

Prompts used in Chatgpt

Prompt 1

I am building a Flask web application called "Career Roadmap Generator".Project Requirements:Flask backend , SQLite database using Flask-SQLAlchemy .API integration will be added later (Gemini/OpenAI) Frontend will be developed separately by my teammate . I am responsible only for the backend .Follow clean Python coding practices and modular structure .Use Python 3.14

Project Idea A user enters a career name (for example: Data Scientist, AI Engineer, Frontend Developer). The application should: Store careers in a database. Store roadmap topics for each career. Store subtopics for each topic. Store generated learning content for each subtopic. Store quizzes for each subtopic. Store user progress.

Important Do NOT generate the entire project at once progress step by step ,Start with Phase 1 only. Explain every file before generating code. Generate code only for database.py , models.py app.py Create the database schema first. Use Flask-SQLAlchemy. Explain how to test that the database is working.

After finishing Phase 1, stop and wait for my confirmation before proceeding to the next phase.

Prompt 2

Phase 1 is complete. Flask application is running successfully. SQLite database is configured using Flask-SQLAlchemy. Models are created: Career Topic SubTopic LearningContent Quiz UserProgress Tables are successfully created in roadmap.db. Database insert and query operations have been tested successfully. Now guide me through Phase 2 only.

Implement roadmap generation and storage logic.

Create a service layer for roadmap generation. Do NOT use Gemini/OpenAI yet. Use mock roadmap data for testing. Create a route where a user provides a career name. Check if the career already exists in the database. If it exists, return the stored roadmap. If it does not exist: Create the career. Generate mock roadmap topics. Store the topics in the database. Return the roadmap. Follow clean architecture and separation of concerns. Explain every file before generating code. Show how to test the route

Phase 2 completed.

Phase3 completed

Phase 4 completed

Phase 5 completed

step is completed. Gemini test successfully returns a response.

All phases are completed and Gemini integration is working.

Convert the current Question + Answer quiz system into a Multiple Choice Question (MCQ) system. Modify the Quiz model.Replace the current answer column with: option_a, option_b, option_c, option_d, correct_answer. Update the Gemini prompt to generate MCQs. Gemini

should return question, 4 options, correct answer. Update the QuizService to Save MCQs to the database. Retrieve MCQs from the database. Progress must NOT increase when an incorrect answer is submitted. Progress should increase only when the correct answer is submitted. A subtopic should be marked completed only when all quiz questions for that subtopic are answered correctly. The existing progress percentage calculation should continue to work.

Generate a project structure.

MANIMAARAN PUGALESWARAN.